Problem Set 1

Zahra Khan

Due: February 11, 2026

Instructions

- Please show your work! You may lose points by simply writing in the answer. If the problem requires you to execute commands in `R`, please include the code you used to get your answers. Please also include the `.R` file that contains your code. If you are not sure if work needs to be shown for a particular problem, please ask.
- Your homework should be submitted electronically on GitHub in `.pdf` form.
- This problem set is due before 23:59 on Wednesday February 11, 2026. No late assignments will be accepted.

Question 1

The Kolmogorov-Smirnov test uses cumulative distribution statistics test the similarity of the empirical distribution of some observed data and a specified PDF, and serves as a goodness of fit test. The test statistic is created by:

$$D = \max_{i=1:n} \left\{ \frac{i}{n} - F_{(i)}, F_{(i)} - \frac{i-1}{n} \right\}$$

where F is the theoretical cumulative distribution of the distribution being tested and $F_{(i)}$ is the ith ordered value. Intuitively, the statistic takes the largest absolute difference between the two distribution functions across all x values. Large values indicate dissimilarity and the rejection of the hypothesis that the empirical distribution matches the queried theoretical distribution. The p-value is calculated from the Kolmogorov- Smirnoff CDF:

$$p(D \leq d) = \frac{\sqrt{2\pi}}{d} \sum_{k=1}^{\infty} e^{-(2k-1)^2\pi^2/(8d^2)}$$

which generally requires approximation methods (see Marsaglia, Tsang, and Wang 2003). This so-called non-parametric test (this label comes from the fact that the distribution of the test statistic does not depend on the distribution of the data being tested) performs poorly in small samples, but works well in a simulation environment. Write an `R` function that implements this test where the reference distribution is normal. Using `R` generate 1,000 Cauchy random variables (`rcauchy(1000, location = 0, scale = 1)`) and perform the test (remember, use the same seed, something like `set.seed(123)`, whenever you're generating your own data).

Before conducting the test, it is crucial to outline the Null and Alternative hypothesis.

Null Hypothesis: The datasets are the same or follow the same, specific distribution. In this case, we are testing if our dataset follows a Normal Distribution.

Alternative Hypothesis: The datasets differ and follow a different distribution. In this case, our dataset does not follow a Normal Distribution.

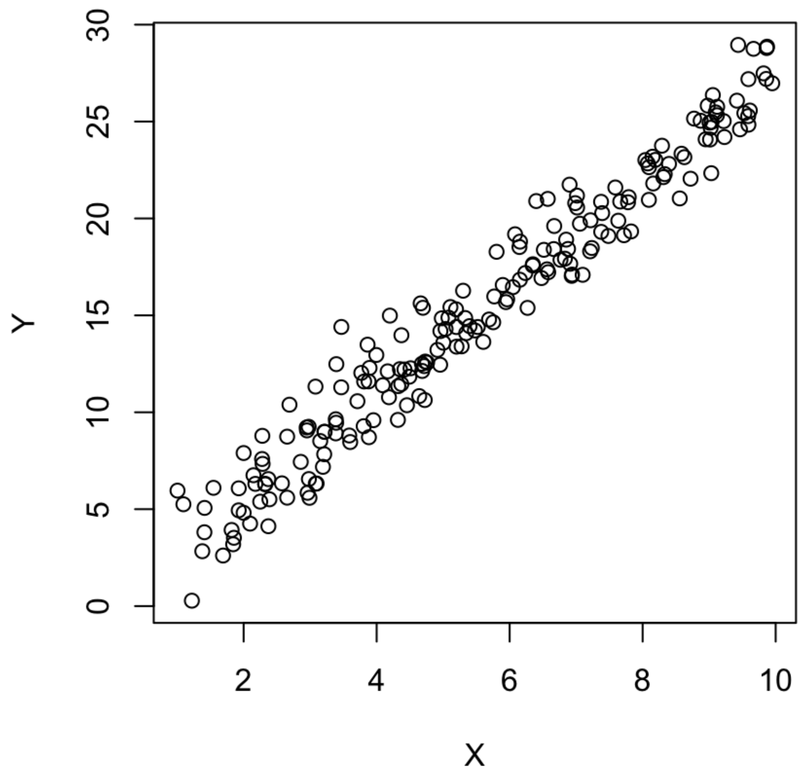
```r
set.seed(123)
data <- (rcauchy(1000, location = 0, scale = 1))
KSpval <- function(data) {
  k <- length(data)
  ECDF <- ecdf(data)
  empiricalCDF <- ECDF(data)
  d_value <- max(abs(empiricalCDF - pnorm(data)))
  i <- 1:k #taking a vectorized approach instead of a loop for calculating the
    terms
  terms<- exp(-(((2*i - 1)^2) * pi^2)/(8*(d_value^2))) #calculating the terms
    for all 100 k's
  d_obs <- sqrt(2 * pi)/d_value * sum(terms) #calculating the observation
return(d_obs)
}
KSpval(data)
# [1] 5.652523e-29
```

After conducting the test, we have obtained a p-value of 5.652523e-29, which, at a threshold of alpha = 0.05, is extremely statistically significant. This means we have sufficient evidence to reject the null hypothesis that our dataset does not follow a normal distribution.

Question 2

Estimate an OLS regression in `R` that uses the Newton-Raphson algorithm (specifically `BFGS`, which is a quasi-Newton method), and show that you get the equivalent results to using `lm`. Use the code below to create your data.

Prior to running the codes I ensured to plot my graph to visualize the data.



After plotting the graph, and inspecting the shape and structure of our data, we've determined to use a log-likelihood function for a normal distribution.

```r
set.seed (123)
data <- data.frame(x = runif(200, 1, 10))
data$y <- 0 + 2.75*data$x + rnorm(200, 0, 1.5)
#Plotting the data:
plot(data$x, data$y, ylab = "Y", xlab = "X")
#creating the linear likelihood function
linear.lik <- function(theta, y, X) { #creating a log of likelihood function
  n <-  nrow(X)
  k <- ncol(X)
  beta <- theta[1:k]
  sigma2 <- theta[k+1]^2
  e <- y - X%*%beta
  logl <- -.5*n*log(2*pi)-.5*n*log(sigma2) - ( (t(e) %*%e)/ (2*sigma2) )
return(-logl)
}
#running with optim.
MLEt <- optim(fn=linear.lik, y=data$y, X = cbind(1, data$x), par=c(1,1,1),
    hessian=T, method ="BFGS")
MLEt$par
#[1]  0.1429829  2.7263116 -1.4423360
linear <- summary(lm(data$y ~ data$x))
#running an OLS model to compare
```

When running the "lm" function in R with the same datapoints, I received an intercept of 0.13919 and a beta of 2.72670. Both of these values are very similar.